

DeltaAttention

1 Introduction

Before the Transformer era, language models were built on recurrent neural networks (RNNs) and their variants such as LSTMs and GRUs. These models were used in small-scale conversational assistants like Siri and Alexa, but their usefulness was limited. RNNs encode context into a fixed-size hidden state, which means they forget information as sequences grow longer and the hidden state gets diluted. More importantly, they process text sequentially token by token, making it impossible to parallelize training across the sequence. This sequential bottleneck kept these models small.

The groundbreaking paper *Attention Is All You Need* [12] changed the field. Transformers process all tokens in a sequence simultaneously, making it possible to train on massive datasets using GPU parallelism. Scaled to billions of parameters on web data, GPT-3 [1] could do in-context learning, translation, coding, and reasoning without any task-specific training. The scale of the data and model allowed a certain amount of generalization that earlier architectures could not achieve.

The mechanism behind this is the attention module. Each token attends to every other token in the sequence simultaneously, allowing the model to capture long-range dependencies. However, computing attention for a sequence of length n requires $O(n^2)$ time and memory, since every token pair must be considered. This quadratic scaling kept context windows short. GPT-3, for instance, was limited to 2048 tokens.

FlashAttention [4] addressed the memory side of this by computing attention in tiles in on-chip SRAM rather than materializing the full attention matrix in HBM, bringing memory use down to linear. On handling token position, the original Transformer used absolute positional encodings — positions were essentially hardcoded, so models simply broke on sequences longer than they were trained on. RoPE [11] encodes position relative to other tokens rather than absolutely, which generalizes much better to longer sequences, and extensions such as YaRN [9] and LongRoPE [5] pushed this further. Together, these advances enabled the jump to context windows of 100k tokens and beyond.

Long context allowed a world of practical applications for LLMs. Coding agents can now take in entire codebases and reason about them. Long documents — legal contracts, research papers, financial reports — can be analyzed in a single pass. Retrieval-Augmented Generation (RAG) feeds retrieved documents directly into the model context, allowing LLMs to answer questions grounded in external knowledge that was not part of their training data. These applications share a common bottleneck. LLM inference consists of two stages: the prefill stage, which processes the entire input prompt in a single forward pass, and the decoding stage, which generates output tokens one by one. As context length grows, the quadratic time complexity of attention makes prefill increasingly expensive. Memory was the original wall; FlashAttention removed it but did not address the time complexity — computing attention on long contexts is slow.

A large body of work aims to accelerate attention for long context tasks by exploiting the empirical sparsity of the attention matrix: most tokens only attend strongly to a small local window and a few distant positions, so many token pair interactions do not need to be computed at all. Early work used fixed sparse patterns [2]; more recent methods determine the pattern dynamically based on the input [6, 7, 16]. All of these methods target sparsity in the attention matrix itself.

In this thesis we propose **DeltaAttention**, a prefill acceleration method that exploits a different source of sparsity: rather than skipping blocks of the attention matrix, it compresses the key matrix K directly before the QK multiplication. Our main contributions are:

- A method to accelerate prefill by compressing K along the sequence dimension before the QK multiplication, reducing the FLOPs required proportionally to the compression ratio.
- Custom Triton kernels that realize this speedup in practice, demonstrating wall-clock acceleration and not just a theoretical reduction in FLOPs.

2 Preliminaries

2.1 Transformer Overview

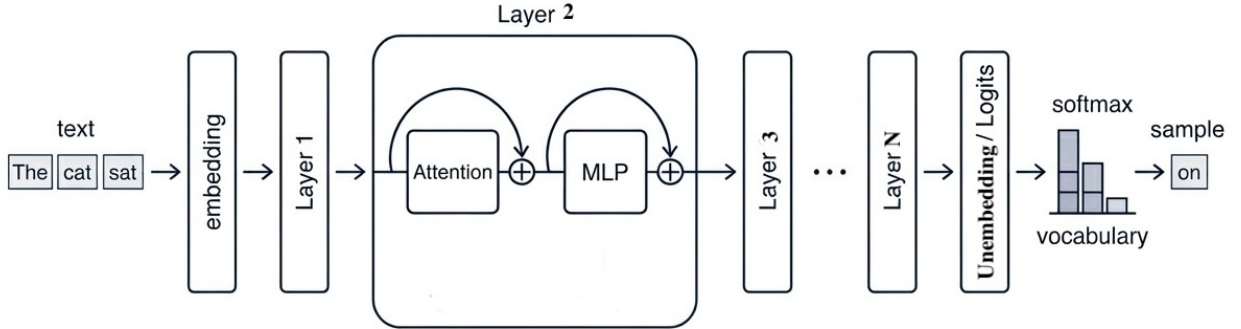


Figure 1: Overview of a Transformer language model. Text is tokenized, each token is looked up in the embedding table, and the resulting vectors pass through N layers, each consisting of an attention module and an MLP. The final embedding of the last token is projected to logits over the vocabulary, a token is sampled, and the process repeats.

A Transformer language model processes text as follows (Figure 1). Text is first split into tokens, and each token is mapped to a vector by looking it up in a learned embedding table. This gives a sequence of vectors of shape $n \times d$, where n is the sequence length and d is the embedding dimension.

These vectors are then passed through a chain of N identical layers. Each layer consists of two modules: an attention module followed by an MLP, each with a residual connection. Through these layers, the embedding of each token is updated to incorporate information from the rest of the sequence.

After the final layer, the embedding of the last token is projected back to the vocabulary size by an unembedding matrix, producing a logit for each vocabulary entry. A softmax turns these logits into a probability distribution, and the next token is sampled from it. This token is appended to

the sequence, and the whole forward pass is run again to generate the token after that. Generation continues until an end-of-sequence token is produced.

2.2 Attention Block

This section explains the inner workings of the attention block shown in Figure 1.

2.2.1 Self-Attention

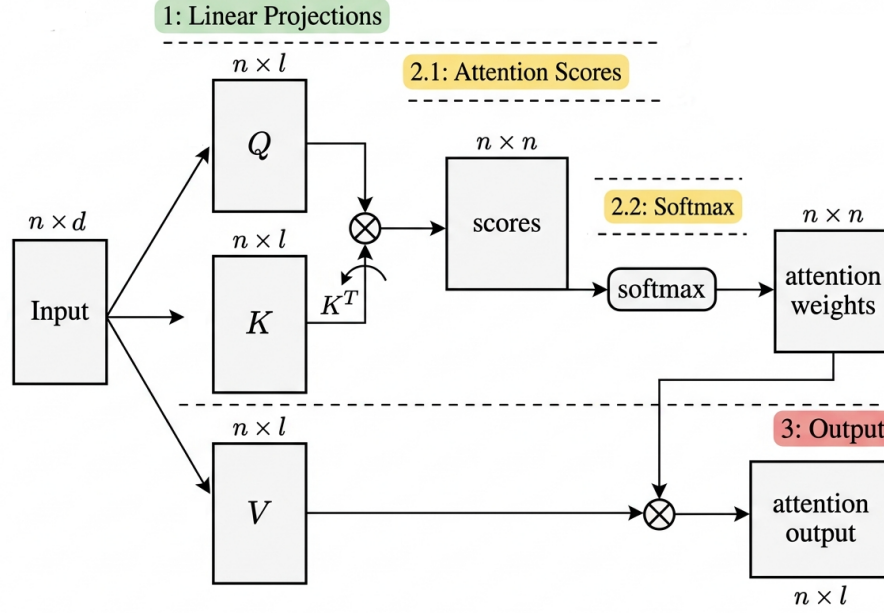


Figure 2: Self-attention mechanism. The input is projected to queries Q , keys K , and values V . The dot product $QK^\top$ gives attention scores, which are normalized by softmax to produce attention weights. The output is a weighted sum of the value vectors.

Self-attention is the operation inside each head (Figure 2). The input hidden state matrix of shape $n \times d$ is projected to three matrices, queries Q , keys K , and values V , each of shape $n \times l$, using learned projection matrices.

The attention scores are computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{l}}\right) V \quad (1)$$

The $QK^\top$ product gives an $n \times n$ matrix where entry (a, b) is the dot product of token a 's query vector with token b 's key vector. Intuitively, the query of a token encodes what it is looking for, and the key of a token encodes what it has to offer. Their dot product measures how relevant token b is to token a . Dividing by $\sqrt{l}$ keeps the dot products from growing too large before the softmax.

Applying softmax row-wise turns each row into a probability distribution, the attention weights, which say how much each token attends to every other token.

The attention weights are then multiplied with the value matrix V to produce the attention output: each token's output is a weighted sum of all value vectors, where the weights come from its row of the attention matrix. Intuitively, the value vector of a token encodes the information it

contributes to other tokens, and the attention weight determines how much of that contribution is picked up.

2.2.2 Multi-Head Attention

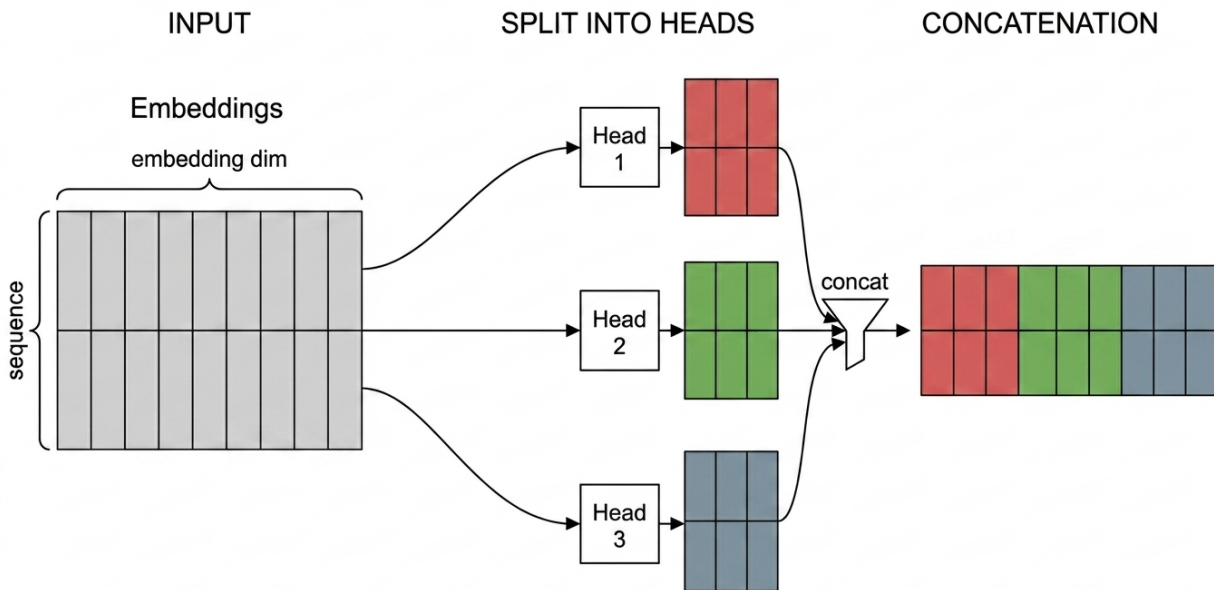


Figure 3: Multi-head attention. The input matrix of shape $n \times d$ is projected by h independent heads, each mapping the full input to a smaller subspace of width $l = d/h$ to produce per-head queries, keys, and values of shape $n \times l$. Each head runs self-attention independently. The outputs are concatenated along the embedding dimension to recover the original $n \times d$ shape.

The attention module inside each Transformer layer is a multi-head attention (MHA) block (Figure 3). Rather than running a single attention computation over the full embedding dimension, MHA runs h attention computations (heads) in parallel, each operating in a smaller subspace of width $l = d/h$. The input to the block is the hidden state matrix of shape $n \times d$, where n is the sequence length and d is the embedding dimension. Each head has its own learned projection matrices that map the full d -dimensional input down to l dimensions, producing per-head queries, keys, and values of shape $n \times l$. Each head sees the full sequence and the full embedding, but through its own projection, different heads facilitate different attention patterns and capture different types of token interactions. The output of each head has the same shape $n \times l$, and the h outputs are concatenated along the embedding dimension to give a final output of shape $n \times d$, matching the input.

2.3 FlashAttention

Materializing the full $n \times n$ attention scores matrix has two critical problems.

The first is memory capacity. The matrix grows quadratically with sequence length: at 10k tokens it is roughly 200 MB per head, at 100k tokens it is around 20 GB per head, and at 1 million

tokens it exceeds 2 TB per head. MHA runs h heads in parallel, so these numbers scale by h , a model like Llama 3 8B uses 32 heads. An H100 has 80 GB of HBM, so at long sequences the scores matrices alone exhaust GPU memory by a large factor.

The second problem is memory bandwidth. Even when the matrix fits, standard attention is bottlenecked by repeated data movement between HBM and the small on-chip SRAM. Computing attention requires multiple passes over the scores: write scores to HBM, read them back for softmax, write the result, read it again for the matmul with V . Each of these round trips is slow, and they dominate the runtime at long sequences.

FlashAttention [4] solves both problems by computing attention in tiles that fit in SRAM. Within each tile, the score computation, softmax, and matmul with V are fused into a single pass, so intermediate results are never written to HBM. This requires an incremental softmax update to accumulate the correct result across tiles without materializing the full scores matrix. The peak memory footprint drops from $O(n^2)$ to $O(n)$, and the intermediate HBM read/writes are eliminated.

2.3.1 FlashAttention Algorithm

Q , K , and V are divided into fixed-size blocks along the sequence dimension (typically 64 or 128 tokens per block). The outer loop iterates over Q blocks; for each Q block, an inner loop iterates over all K and V blocks in sequence.

For each (Q block, K/V block) pair, three steps happen in SRAM:

- The Q block and K block are multiplied to produce an attention scores tile of shape $B_q \times B_k$, where B_q is the query block size and B_k is the key block size.
- Online softmax is applied to the scores tile. Standard softmax requires the full row to normalize, but online softmax maintains a running maximum and a running sum that are corrected as each new tile arrives, producing numerically identical results without having the full row in memory.
- The attention weights tile is multiplied with the V block to get a partial attention output of shape $B_q \times d$. This is added to a running output accumulator for the current Q block, which collects the contributions from all K/V blocks seen so far.

Once all K/V blocks have been processed, the running accumulator holds the sum of all partial contributions. It is rescaled by the final softmax normalizer from the online softmax and written back to HBM as the attention output for this Q block.

FlashAttention 1 parallelizes this computation over the batch and head dimensions. FlashAttention 2 [3] additionally parallelizes over Q blocks: each kernel instance handles one Q block independently, iterating over all K and V blocks on its own. Since there are no dependencies between Q blocks, this is feasible and enables significantly more parallelism.

2.3.2 FlashAttention Demonstration

Figure 4 shows a FlashAttention 2 program computing the second query block (Q_2), with block size 2, sequence length 8, and head dimension 1. Other programs handle the remaining Q blocks in parallel. The gray upper-triangular blocks of S mark KV blocks skipped entirely: every token in those blocks lies in the future of the corresponding query block.

In iteration 1 (left), Q_2 and K_1 are loaded into SRAM and multiplied to produce scores tile S_{21} . After online softmax, S_{21} is multiplied with V_1 and accumulated into O_2 .

In iteration 2 (right), the K and V blocks advance to block 2. Q_2 and K_2 produce tile S_{22} , which is multiplied with V_2 and again accumulated into O_2 . The program stops here: blocks K_3 and beyond lie entirely in the future relative to Q_2 , so they are skipped due to causality.

After all K/V blocks have been processed, O_2 holds the correct attention output for the tokens in Q_2 , rescaled by the final softmax normalizer.

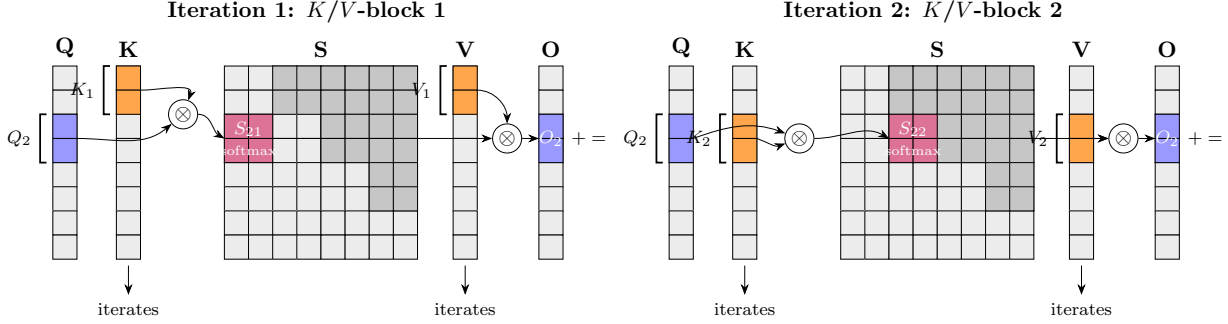


Figure 4: Two consecutive inner-loop iterations for Q -block 2. Left: K_1 and V_1 produce tile S_{21} , accumulated into O_2 . Right: K_2 and V_2 produce tile S_{22} , accumulated into O_2 .

2.4 Local Attention Window

Attention in LLMs is empirically sparse: tokens strongly attend to their neighbors and a few distant tokens. Xiao et al. [15] identified two regions that receive strong attention. The first is a narrow band along the diagonal, the **critical diagonal**, meaning tokens attend mostly to their neighbors. The second is the first few tokens of the sequence, the attention sink.

Figure 5 shows this for two layers of LLaMA 3.2. Both layers show the same structure: high attention weights on the diagonal and on the initial token.

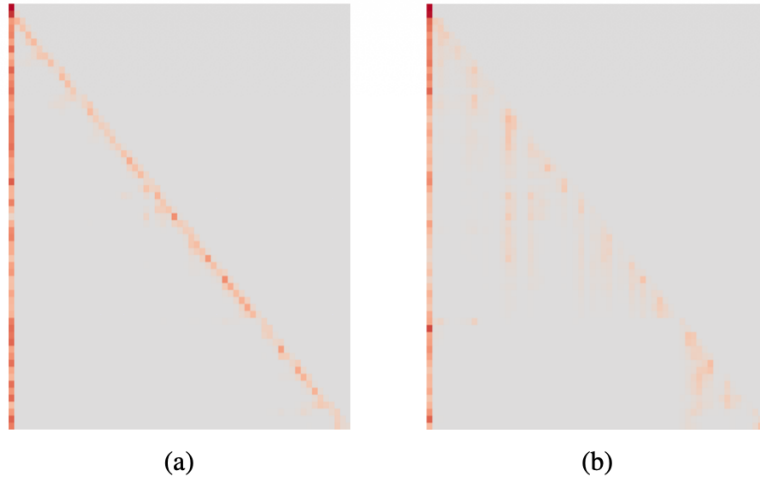


Figure 5: Heatmap of attention scores in two layers of LLaMA 3.2 during prefill. Figure reproduced from Qi et al. [10].

This observation motivates computing the attention exactly for tokens in the critical diagonal in prefill acceleration methods. Jiang et al. [6] have their A-shape attention pattern. Qi et al.

[10] maintain an exact window for the most recent tokens while compressing the rest via delta key vectors. DeltaAttention follows the same principle: the critical diagonal is processed with the full uncompressed keys and values.

3 Related Work

Attention complexity and FlashAttention. Self-attention scales quadratically in both time and memory with sequence length, which made long-context inference impractical for early Transformer models. FlashAttention [4] addressed the memory side by computing attention in tiles in on-chip SRAM, avoiding materializing the full attention matrix in HBM. This brought memory use down to linear and cut HBM traffic substantially, making attention practical for sequences up to tens of thousands of tokens. However, FlashAttention is exact—every attention score is still computed—so the quadratic time complexity remains. As sequences grow attention computation becomes the dominant runtime cost and approximate methods become necessary.

Sparse attention for prefill. A natural way to reduce the time cost is to exploit the fact that attention matrices are empirically sparse: most tokens attend strongly to only a small fraction of others. Child et al. [2] were among the first to demonstrate this, observing that trained Transformer attention matrices concentrate on local windows and a few distant positions. They proposed fixed sparse patterns—local windows combined with strided or fixed summary positions—to approximate full attention with fewer FLOPs. The limitation is that a fixed pattern cannot adapt to the content of a given input.

This motivated a shift to dynamic sparse attention, where the sparsity pattern is determined on the fly. Jiang et al. [6] found that LLM attention heads exhibit consistent structural patterns across different inputs—A-shape, Vertical-Slash, and Block-Sparse—and assigned each head its dominant pattern offline, building sparse indices at inference time accordingly. SpargeAttn [17] is a parallel work that takes a more general approach designed to work across diverse model types beyond language models. It selects a representative per block and uses representative-level attention scores to predict which block pairs are near-zero and can be skipped entirely, without assuming any fixed pattern. FlexPrefill [7] and XAttention [16] are more recent works that push the state of the art further. FlexPrefill adapts both the sparsity pattern and compute budget dynamically per head using Jensen-Shannon divergence to classify the pattern type at runtime. XAttention uses the sum of antidiagonal values within each attention block as a lightweight proxy for block importance, achieving aggressive pruning at minimal overhead.

All of these methods target sparsity in the attention matrix at the block level. Skipping entire blocks maps cleanly onto FlashAttention’s block-based execution: if a block is not selected, its tile is simply never computed.

As a parallel line, Qi et al. [10], a concurrent work from our group at LIACS, targets a different kind of sparsity. While the methods above exploit the empirical sparsity of the attention map, they observe that adjacent key vectors are similar, so encoding each key as a delta from the previous one yields a sparse delta key matrix. They then propose a method to compute attention directly using this delta key matrix, reducing the overall computation. However, the delta is applied element-wise, giving unstructured sparsity scattered throughout the delta matrix—a pattern that GPU hardware cannot exploit efficiently, requiring custom hardware to accelerate.

KV cache compression for decoding. A separate line of work targets the memory bottleneck of autoregressive decoding. Decoding is memory-bound: at every generation step the full KV

cache must be loaded from HBM, so the KV cache dominates both memory capacity and memory bandwidth. As context length grows, the cache can exceed GPU memory entirely, but even when it fits, a smaller cache directly translates to faster decoding by reducing the amount of data loaded per step. Zhang et al. [19] made the foundational observation that a small set of “heavy hitter” tokens receives the majority of attention mass, and proposed dynamically retaining only those tokens while evicting the rest. Li et al. [8] refined this with a query-aware eviction policy: each attention head observes which tokens are attended to within an observation window at the end of the prompt, and retains only those positions for subsequent decoding. Zhang et al. [18] pointed out that evicting tokens causes irreversible information loss, and proposed merging evicted tokens into their neighbors rather than discarding them. Wang et al. [13] arrived at merging from a different angle: they observed that adjacent token embeddings in the key matrix exhibit high cosine similarity within a sequence, and that this pattern is consistent across datasets and layers. This motivates merging consecutive similar key tokens into a single representative, compressing the cache without throwing away information.

InfLLM [14] spans both prefill and decoding. It offloads the KV cache to CPU memory and selectively loads blocks to the GPU based on predicted attention relevance, using a block-representative scheme to decide which blocks a given query is likely to attend to. Since CPU RAM is far more abundant than GPU VRAM, this effectively extends the usable context window without requiring the full cache on device at once.

DeltaAttention. DeltaAttention is a prefill acceleration method. Its starting point is the same observation as Qi et al. [10]: taking deltas between adjacent key vectors reveals sparsity in K . We asked whether that sparsity could be made *structured*. Applying the delta at the token level rather than element-wise means that entire rows of K with small delta magnitude can be dropped. This produces structured sparsity along the sequence dimension: K can be physically compressed before the QK multiplication, reducing FLOPs in proportion to the number of pruned rows.

After arriving at this formulation, we noticed it converges with the core observation behind KVMerger [13]: adjacent key token embeddings are similar. KVMerger uses this to compress the KV cache for decoding; we use the same insight to compress K at prefill. DeltaAttention is, to our knowledge, the first work to apply this insight as a prefill optimization with structured, GPU-friendly sparsity.

4 Methodology

4.1 (1) Pre-processing

The pre-processing stage takes the key matrix K as input and outputs a binary keep mask of length n . It analyzes K row by row to decide which rows to keep and which to drop (steps 1.1 and 1.2 in Figure 6).

We initialize a keep mask of length n to all ones, meaning all rows are initially kept. The anchor is set to the first row K_0 . We then iterate over the remaining rows:

1. Compute the Euclidean distance from row K_i to the current anchor:

$$\delta_i = \|K_i - \text{anchor}\|_2 \tag{2}$$

2. If $\delta_i > \tau$ for some threshold τ : the row is sufficiently different from the anchor, so we keep it and move the anchor to K_i .

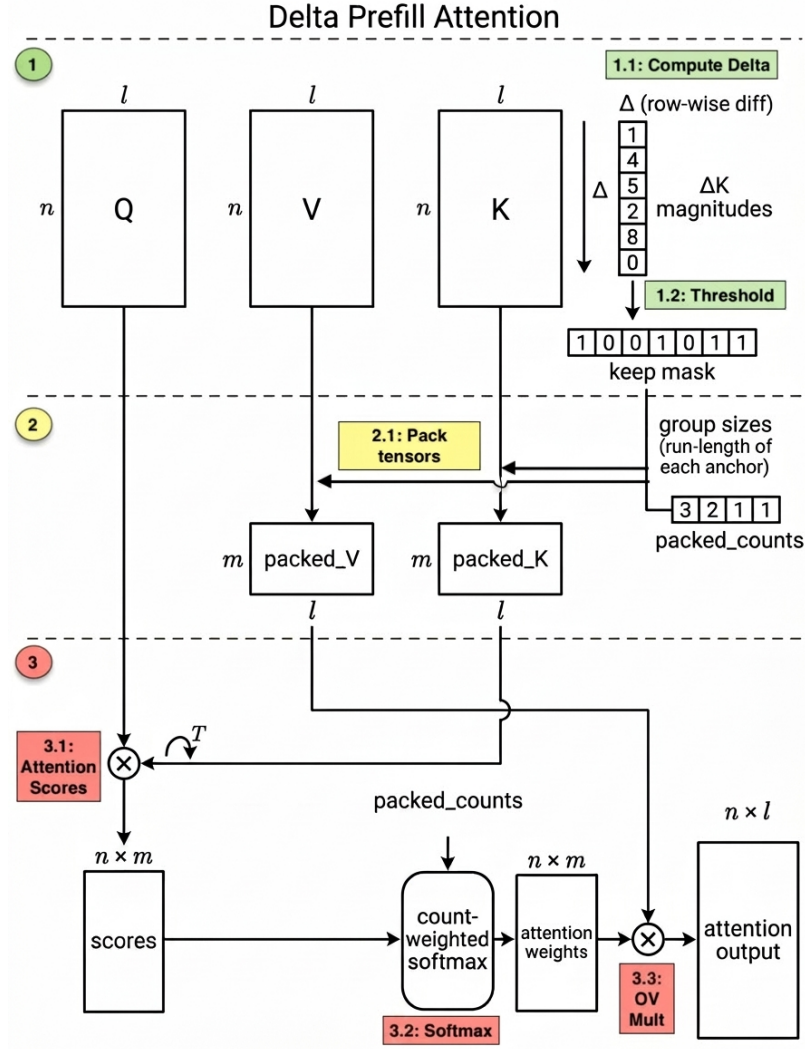


Figure 6: ...

3. If $\delta_i \leq \tau$: the row is similar enough that the anchor can represent it, so we drop it by setting $\text{keepmask}[i] = 0$ and leave the anchor unchanged.

The output keep mask has a 1 at each anchor position and a 0 at each dropped position.

4.2 (2) Packing

The packing stage takes the keep mask from stage (1) as input and outputs compressed key and value matrices $\hat{K}$ and $\hat{V}$, and a vector packed_counts (step 2.1 in Figure 6).

We drop all rows of K and V where the keep mask is 0, keeping only the anchor rows. The resulting $\hat{K}$ and $\hat{V}$ have shape $m \times l$, where $m < n$ is the number of anchors.

We also compute packed_counts of length m , where $\text{packed_counts}[j]$ is the run length of anchor j , the number of rows it represents including itself.

4.3 (3) Self-Attention

The final stage takes the original query matrix Q and the packed matrices $\hat{K}$, $\hat{V}$, and `packed_counts` from stage (2) as input, and outputs the attention output of shape $n \times l$. It follows the same steps as standard self-attention (Section 2.2.1), with two differences: Q is multiplied against the compressed $\hat{K}$ and $\hat{V}$ instead of the full matrices, and the softmax is weighted by `packed_counts`.

1. **Attention scores** (step 3.1): Multiply Q by $\hat{K}^\top$ to get the score matrix:

$$S = \frac{Q\hat{K}^\top}{\sqrt{l}} \quad (3)$$

S has shape $n \times m$, where entry (i, j) encodes how much token i attends to anchor j .

2. **Count-weighted softmax** (step 3.2): An anchor representing many tokens should have a proportionally larger effect than one representing a single token. To account for this, we scale each column of S by the corresponding entry in `packed_counts` before applying the softmax:

$$W_{i,j} = \frac{\text{packed_counts}[j] \cdot \exp(S_{i,j})}{\sum_{k=1}^m \text{packed_counts}[k] \cdot \exp(S_{i,k})} \quad (4)$$

For example, if token i attends to anchor j with score 5 and anchor j represents 10 tokens, the score is updated to 50 before the softmax. This is equivalent to running standard self-attention where the anchor key vector is copied to all token positions it represents.

3. **Attention output** (step 3.3): Multiply the attention weights by the packed value matrix:

$$\text{output} = W\hat{V} \quad (5)$$

The compressed sequence dimension m disappears after this multiplication, giving an output of shape $n \times l$, identical in shape to the output of standard self-attention.

4.4 Flash Attention Integration

To test DeltaAttention on long context tasks we modified Flash Attention 2 to operate on the compressed $\hat{K}$ and $\hat{V}$ matrices. The kernel follows the same structure as FlashAttention 2 (Section 2.3.1): an outer loop over query blocks, with each kernel program handling one query block independently. The inner loop is modified to operate in two phases.

Alongside $\hat{K}$, $\hat{V}$, and `packed_counts`, we store a `packed_timestamps` vector of length m . Entry j holds the index in the original sequence of the last token that anchor j represents. For example, if anchor j represents tokens 5, 6, and 7 from the original sequence, then `packed_timestamps[j] = 7`. This vector can be computed as a cumulative sum of `packed_counts`, minus one. It lets the kernel track how far into the original sequence each anchor’s coverage extends.

Phase 1: Compressed history. The first phase iterates over blocks of $\hat{K}$ and $\hat{V}$ instead of blocks of K . For each packed block, the kernel reads the `packed_timestamps` of the anchors in that block and takes their maximum. This is the reach of the block: the furthest position in the original sequence that any anchor in the block represents.

We define the **critical diagonal** of a query block as the span of positions the block itself covers, `[query_block_start, query_block_end]`. Any anchor whose reach falls within this span represents tokens that overlap with the current query block and must be processed exactly. This follows the

local attention window principle described in Section 2.4: tokens attend most strongly to their immediate neighbors, so those neighbors must be kept exact.

If the reach falls before the critical diagonal ($\text{reach} < \text{query_block_start}$), the block is safe to process in compressed form. The kernel computes the attention scores against the packed key vectors and applies count-weighted softmax exactly as in Equation 4, updating the online softmax running state (m_i, l_i, acc) as in standard FlashAttention.

If the reach falls inside or past the critical diagonal, the first phase stops. Those anchors cover tokens that belong to the current query block, so they need to be processed exactly.

Phase 2: Critical diagonal. The second phase switches to the original uncompressed K and V . Starting from just after the last safe anchor’s reach and going up to the end of the current query block, it runs standard FlashAttention over the exact token vectors.

References

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [2] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [3] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, 2024.
- [4] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359, 2022.
- [5] Yiran Ding, Li Lyna Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. LongRoPE: Extending LLM context window beyond 2 million tokens. *arXiv preprint arXiv:2402.13753*, 2024.
- [6] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *Advances in Neural Information Processing Systems*, 2024.
- [7] Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. FlexPrefill: A context-aware sparse attention mechanism for efficient long-sequence inference. In *International Conference on Learning Representations*, 2025.
- [8] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.
- [9] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023.

- [10] Jiawen Qi, Chang Gao, Zhaochun Ren, and Qinyu Chen. DeltaLLM: A training-free framework exploiting temporal sparsity for efficient edge LLM inference. *TODO: add arXiv ID or venue*, 2024.
- [11] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [13] Zheng Wang, Boxiao Jin, Zhongzhi Yu, and Minjia Zhang. Model tells you where to merge: Adaptive KV cache merging for LLMs on long-context tasks. *arXiv preprint arXiv:2407.08454*, 2024.
- [14] Chaojun Xiao, Pengl Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. InfLLM: Training-free long-context extrapolation for LLMs with an efficient context memory. *arXiv preprint arXiv:2402.04617*, 2024.
- [15] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, 2024.
- [16] Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. XAttention: Block sparse attention with antidiagonal scoring. *arXiv preprint arXiv:2503.16428*, 2025.
- [17] Jintao Zhang, Chendong Xiang, Haofeng Huang, Jia Wei, Haocheng Xi, Jun Zhu, and Jianfei Chen. SpargAttention: Accurate and training-free sparse attention accelerating any model inference. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*. PMLR, 2025.
- [18] Yuxin Zhang, Yuxuan Du, Gen Luo, Yunshan Zhong, Zhenyu Zhang, Shiwei Liu, and Rongrong Ji. CaM: Cache merging for memory-efficient LLMs inference. *TODO: add arXiv ID or venue*, 2024.
- [19] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023.